ОБОСНОВАНИЕ ПРОЕКТА

Расширенная профессиональная редакция: актуальность, практическая нужность, полезность узкой области EH/Unwind, динамический слой, Ghidra-интеграция и методология тестирования/валидации.

Параметр	Содержание
Проект	EH/Unwind Analyzer - инструмент структурного анализа и динамической валидации Windows x64 PE32+ AMD64 EH/Unwind-метаданных.
Главная защита	Проект нужен не массовому пользователю, а экспертам, которые анализируют PE-файлы глубже уровня строк, импортов и общей дизассемблерной картины. Узкая область является осознанной специализацией.
Что добавлено в этой редакции	Расширено профессиональное обоснование, добавлена подробная методология тестирования: виды тестов, число бинарников, роль synthetic/real corpus, критерии достаточности и ответы комиссии.

1. Короткая защитная позиция

EH/Unwind Analyzer следует защищать не как универсальный анализатор вредоносных программ, а как специализированный инженерный инструмент для проверки одного сложного и важного слоя Windows x64: exception handling and stack unwinding metadata. Его область - PE32+ AMD64, Exception Directory, .pdata, RUNTIME_FUNCTION, .xdata, UNWIND_INFO, handler RVA, chained unwind info, section placement, reuse/dominance patterns and optional runtime evidence.

Ключевая мысль для комиссии: проект узкий, но не искусственный. Windows x64 реально использует metadata-driven unwinding. Это значит, что границы функций, восстановление caller context, обработка исключений, stack walking, debugging and runtime-generated code support зависят от корректности этих таблиц. Если metadata повреждена или необычно устроена, реверсер получает риск неверно понять стек, обработчики, диапазоны функций и exception-based routes.

Проект автоматизирует ручную работу, которая иначе выполнялась бы в PE viewer, Ghidra/IDA, hex editor and debugger: найти Exception Directory, проверить .pdata, прочитать RUNTIME_FUNCTION entries, перейти к UNWIND_INFO, учесть CountOfCodes, дополнительные slots, alignment, handler tail, chained info, section permissions, suspicious placement, reuse and handler dominance. Поэтому ценность проекта - не в лозунге “он всё ловит”, а в том, что он делает повторяемым и проверяемым трудоёмкий low-level анализ.

Профессиональная формулировка: EH/Unwind Analyzer строит проверяемую карту EH/Unwind-инфраструктуры конкретного PE-файла, фиксирует структурные нарушения и необычные patterns, разделяет observation, finding, severity, confidence and verdict, а при dynamic validation собирает runtime evidence по исключениям, handler hits and dynamic function tables. Это инструмент triage and evidence preparation for human reverse engineering.

> Фраза для защиты: “Я не называю проект антивирусом и не утверждаю, что он доказывает вредоносность. Я показываю специализированный анализатор низкоуровневой Windows x64 EH/Unwind-инфраструктуры, который помогает реверсеру быстрее и надёжнее перейти к ручной проверке”.

2. Почему узкая тема имеет практическую ценность

Вопрос “зачем нужен узкий инструмент” нужно отвечать через экономику экспертной работы. В reverse engineering есть много узких инструментов: дизассемблерные плагины, symbolizers, crash dump utilities, stack unwind helpers, PE validators, fuzzer harnesses, YARA rule tools. Они не рассчитаны на массового пользователя, но в своей профессиональной зоне резко уменьшают время анализа и снижают число ошибок.

EH/Unwind metadata - это именно такая зона. На первый взгляд .pdata and .xdata выглядят как служебные таблицы. Но на Windows x64 они находятся между PE Format, ABI, compiler output, exception dispatch, debugging and runtime-generated code. Маленькая ошибка в интерпретации адреса, размера или флага может дать крупную ошибку в выводе: неверный handler target, ложный parse failure, неправильное восстановление stack frame, игнорирование dynamic function table или неверный вывод о странности файла.

Практическая полезность проекта проявляется не в том, что он заменяет реверсера. Он делает реверсера быстрее и аккуратнее. Инструмент берёт на себя массовую структурную проверку и отдаёт человеку уже организованную evidence-карту: какие ranges есть, какие UNWIND_INFO читаются, какие записи сломаны, где handler target, какие секции используются, какие findings появились, какова confidence, что нужно проверить вручную.

Сценарий	Земная польза
Большой PE с тысячами функций	Ручной обход .pdata/.xdata становится дорогим. Анализатор быстро выделяет записи с нарушениями, подозрительными placements, parse errors and reuse/dominance patterns.
Protected/packed sample	Static может увидеть необычную EH/Unwind-карту, а dynamic - зафиксировать handler hits или runtime function table activity. Это помогает понять, где механизм защиты может использовать exceptions/unwind.
Crash/debug case	Повреждённая или странная unwind metadata объясняет, почему stack walking, caller context или exception dispatch могут быть ненадёжными.
Командная работа	JSON/HTML/CSV/Ghidra CSV позволяют передавать не мнение исследователя, а воспроизводимый report с hash, settings, findings and evidence.
Обучение и ВКР	Проект показывает, что теория Windows x64 ABI/PE/EH превращена в работающий инструмент с тестами и отчётами.

3. Предмет, объект и границы исследования

Объект исследования - Windows PE32+ AMD64 binaries and their exception/unwind metadata. Предмет исследования - методы безопасного чтения, структурной валидации, декодирования, эвристической оценки и динамической проверки EH/Unwind-инфраструктуры таких файлов.

Это разграничение важно. Проект не исследует все аспекты вредоносного ПО: persistence, network C2, injection, cryptography, anti-debug, filesystem behavior, privilege escalation. Он исследует конкретный слой: как PE-файл и Windows runtime описывают функции, stack frames, handlers and function tables. Такая граница делает проект защищаемым: можно точно сказать, что входит в scope, что не входит и почему.

Внутри scope находятся: PE headers, sections, RVA/file offset mapping, Exception Directory, .pdata, RUNTIME_FUNCTION entries, range validity, sorting, overlaps, duplicates, UnwindInfoAddress mapping, UNWIND_INFO fields, unwind code slots, padding, handler flags, chained info, section permissions, metadata placement, reuse, handler dominance, JSON/HTML/CSV/Ghidra reporting, Windows Debug API runtime evidence.

Вне scope текущей версии находятся: полноценное дизассемблирование функции и доказательное сопоставление каждой UWOP operation с реальными bytes пролога; универсальный parser language-specific handler data для всех runtime; автоматическая deobfuscation; malware classification; обход anti-debug; безопасный запуск вредоносов без внешней isolated VM. Эти ограничения не делают проект неполным в заявленном scope. Они показывают, что границы сформулированы честно.

4. Техническое основание актуальности

Техническая актуальность проекта опирается на официальный Windows x64 contract. В 64-битной Windows функции, требующие корректного unwinding, описываются function table entries. Эти entries указывают на unwind information, где закодированы эффекты пролога: изменение RSP, сохранение nonvolatile registers, frame pointer model, handlers or chained info. Это не декоративная отладочная метаинформация. Это данные, которыми пользуются exception handling, unwinder, debugger support and stack walking.

PE Format задаёт container layer: headers, sections, data directories, RVAs. Exception Directory указывает на table of function entries, обычно размещённую в .pdata. x64 exception handling задаёт meaning layer: RUNTIME_FUNCTION and UNWIND_INFO. Dynamic function table APIs задают runtime layer: для dynamically generated code процесс обязан предоставить function table операционной системе. Debug API задаёт observation layer: debugger получает events and exceptions through documented event loop.

Из этих фактов следует практическое требование: анализатор должен не просто печатать поля, а проверять переходы между слоями. File offset не равен RVA. RVA не равен VA. Name .pdata не заменяет DataDirectory. CountOfCodes считает slots, а не “операции”. Handler RVA не является language-specific data. Static file map не равен runtime process map. Именно эти переходы проект и делает явными.

Технический уровень	Почему важен	Как покрывается проектом
PE image model	Без корректной карты секций невозможно безопасно читать .pdata/.xdata.	pe.c строит модель секций, image size, headers, RVA mapping and validation counters.
RUNTIME_FUNCTION	Диапазоны функций и UnwindInfoAddress являются основой unwind map.	unwind.c читает entries, checks count, ranges, sorting, overlaps, duplicates.
UNWIND_INFO	Описывает stack/frame effects, handlers and chain info.	decoder учитывает fields, UWOP slots, parse failures, anomalies and summaries.
Heuristic evidence	Parser success не означает trust. Нужна оценка unusual/malformed patterns.	heuristics.c строит findings по evidence families and confidence rules.
Runtime evidence	Dynamic function tables and handler hits появляются только в процессе.	dynamic_win.c собирает exceptions, handler hits, VEH/VCH/UEF, RtlAdd/RtlInstall events.
Reporting	Без отчёта результат нельзя проверить и передать.	report.c создаёт JSON/HTML/CSV/Ghidra outputs and assessment.

5. Почему проект не является просто PE parser

Parser - это необходимый, но нижний слой. Он отвечает на вопрос “можно ли прочитать структуру”. Analyzer отвечает на вопрос “что эти структуры означают для задачи EH/Unwind review и где нужна ручная проверка”. Если программа только печатает BeginAddress, EndAddress and UnwindInfoAddress, это dumper. Если она проверяет границы, сопоставляет sections, декодирует UNWIND_INFO, строит findings, разделяет severity/confidence, пишет отчёты and correlates static/dynamic evidence, это уже система анализа.

EH/Unwind Analyzer должен называться системой анализа, потому что он имеет несколько уровней интерпретации. Он не только читает PE, но и проверяет integrity таблиц, unusual placement, malformed unwind, reuse, handler dominance, dynamic runtime activity and report semantics. Он сохраняет не только raw values, но и причинно-следственные объяснения: почему факт важен, насколько он серьёзен, насколько анализатор уверен и что должен сделать человек дальше.

Parser	Analyzer
Читает структуру и выводит поля.	Строит модель, проверяет корректность, формирует evidence and assessment.
Может не иметь понятия severity/confidence.	Разделяет finding severity, confidence, score, verdict and manual review.
Не обязан иметь dynamic mode.	Имеет optional dynamic validation для runtime evidence.
Не обязан иметь отчётный контракт.	Имеет JSON schema, HTML dashboard, CSV, Ghidra CSV and validation checks.
Не отвечает на вопрос “что проверить дальше”.	Формулирует manual validation steps and representative RVAs.

6. Зачем нужна динамическая часть

Динамическая часть нужна из-за фундаментального различия между PE-файлом на диске и процессом в памяти. Static analysis отвечает на вопрос: “какую EH/Unwind-карту заявляет файл?”. Dynamic validation отвечает на вопрос: “какие exception/unwind-related события реально появились в данном запуске?”. Эти вопросы не заменяют друг друга.

Static не может увидеть уже созданную runtime table, потому что до запуска её ещё нет. Он может только увидеть предпосылки: imports, suspicious placement, handler patterns, references to runtime APIs. Dynamic может увидеть фактические events: exception debug events, handler hits, AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable, RtlInstallFunctionTableCallback. Поэтому dynamic слой нужен для runtime evidence, а не для украшения отчёта.

Важно защищать dynamic честно. Один запуск не покрывает все ветки. Если target требует input, network, UI, elevated rights or delayed condition, dynamic session может не пройти нужный route. Если session timed out, отсутствие handler hit не означает, что handler никогда не сработает. Поэтому report должен сохранять enabled, ran, ok, timed_out, target_exited, event counters, dropped events, breakpoint failures and instrumentation completeness.

Dynamic-задача	Что доказывает	Чего не доказывает
Handler hit	Handler действительно получил управление в конкретном запуске.	Не доказывает, что этот путь всегда выполняется или является вредоносным.
RtlAddFunctionTable	Процесс регистрировал dynamic function table.	Не доказывает автоматически JIT или malware: это может быть нормальный runtime.
First/second chance exceptions	Исключения реально возникали и проходили через debug event flow.	Не доказывает весь control flow и не покрывает неисполненные ветки.
VEH/VCH/UEF events	Процесс использовал runtime exception-handling mechanisms.	Не объясняет handler data без дополнительного ручного анализа.
Timeout/incomplete	Анализатор честно ограничил выполнение.	Не означает отсутствие подозрительного поведения.

Формулировка для защиты: “Dynamic validation в моём проекте нужна не для замены static, а для фиксации runtime evidence. Static строит карту обещаний файла, dynamic проверяет отдельные маршруты живого процесса”.

7. Зачем нужна Ghidra-интеграция

Ghidra-интеграция нужна потому, что reverse engineering не заканчивается чтением JSON. JSON нужен backend, тестам и автоматической обработке. HTML нужен человеку для обзорного чтения. Но реверсер основную работу делает в среде анализа: смотрит disassembly, decompiler, functions, xrefs, comments, bookmarks and bytes. Поэтому результат анализатора должен быть перенесён туда, где человек продолжает проверку.

Ghidra CSV является мостом между автоматическим анализом и ручной экспертизой. Он позволяет перенести function ranges, unwind comments, warnings and representative RVAs в базу Ghidra. Это снижает трение: специалист не копирует адреса из отчёта вручную, а видит подсказки рядом с функцией. Там же он может проверить, похож ли handler на code, соответствует ли section context выводам, и есть ли рядом необычные branch/exception patterns.

Важно: Ghidra-интеграция не означает, что проект заменяет Ghidra. Это наоборот признак профессионального workflow. Analyzer делает узкую автоматическую работу, Ghidra остаётся местом глубокой ручной интерпретации. Такое разделение реалистично и хорошо защищается перед комиссией.

Без Ghidra bridge	С Ghidra bridge
Результат живёт отдельно в HTML/JSON.	Факты анализа попадают в рабочую среду реверсера.
RVA нужно вручную переносить в листинг.	Representative RVA становится точкой навигации.
Findings труднее связать с кодом.	Comment/bookmark помогает смотреть finding рядом с disassembly.
Автоматический анализ ощущается отдельным отчётом.	Он становится частью manual reverse engineering workflow.

8. Уникальность и научно-инженерная новизна

Новизну нельзя формулировать как “никто в мире не умеет читать .pdata”. Это было бы неверно и легко оспоримо. Правильная формулировка: проект систематизирует и реализует специализированный evidence-based pipeline для EH/Unwind metadata Windows x64, объединяя безопасный PE parser, unwind decoder, heuristic findings, dynamic validation, reporting contract, Ghidra bridge and validation methodology.

Научно-инженерная новизна находится в сборке и дисциплине. Официальная документация описывает структуры. Дизассемблеры показывают код. Отладчики показывают runtime events. Но данный проект строит отдельную проверяемую модель: от untrusted bytes к PE image, от PE image к RUNTIME_FUNCTION, от RUNTIME_FUNCTION к UNWIND_INFO, от decoded facts к findings, от findings к assessment, от static map к dynamic evidence, от internal structures к JSON/HTML/Ghidra artifacts.

Для ВКР это сильная позиция: работа не изобретает Windows заново, а строит инструментальный слой между официальным контрактом Windows и практикой reverse engineering. Ценность в том, что ручные знания превращены в проверяемый продукт с тестами, отчётами и документацией.

9. Почему не нейросеть

Нейросеть здесь не является лучшим ядром, потому что основные проверки формальны. Кратность Exception Directory размеру RUNTIME_FUNCTION, выход RVA за SizeOfImage, CountOfCodes, padding, flags, CHAININFO, section permissions and handler RVA placement вычисляются детерминированно. Для таких условий neural network не добавляет надёжности, а снижает объяснимость.

Вторая причина - доказательность. Комиссия и реверсер должны видеть, из какого поля появился finding. “Модель решила” здесь слабее, чем “UnwindInfoAddress не мапится в readable section” или “CountOfCodes требует больше slots, чем доступно”. Проект работает с evidence chain, а не с непрозрачной вероятностью.

Третья причина - отсутствие корректного labeled corpus именно для EH/Unwind anomalies. Чтобы честно обучать ML, нужны сотни или тысячи размеченных samples, где label относится не к общей вредоносности, а к конкретным EH/Unwind нарушениям. Без такого corpus нейросеть была бы демонстрационным украшением, а не научно обоснованным методом.

Корректная позиция: ML можно рассматривать в будущем для кластеризации unusual patterns или приоритизации большого corpus, но ядро structural validation должно оставаться deterministic and explainable.

10. Границы честности: что проект не должен обещать

Профессиональная защита сильнее, если ограничения названы до того, как комиссия сама их найдёт. Проект не является malware classifier, не выдаёт вероятность вредоносности, не заменяет Ghidra/IDA/WinDbg, не восстанавливает полный control flow, не выполняет универсальный parser всех language-specific handler data, не доказывает отсутствие runtime behavior при одном dynamic run и не должен запускать untrusted samples вне isolated VM.

Это не признание провала. Это правильное scope control. Инструмент завершён как EH/Unwind structural analyzer and dynamic evidence collector в заявленной области. Более широкие возможности - prolog-vs-unwind disassembly matching, deeper handler data interpretation, memory snapshot analysis, anti-debug-aware sandboxing - являются развитием, а не обязательным условием базовой полезности.

На защите нельзя говорить “score 70 означает 70 процентов malware”. Правильно: score - bounded engineering weight по правилам проекта; confidence - доверие к интерпретации evidence; severity - серьёзность конкретного finding.

11. Профессиональная методология тестирования

Тестирование такого проекта нельзя сводить к вопросу “сколько exe прогнали”. Один большой набор бинарников без понятной роли хуже, чем меньший, но правильно размеченный corpus. Нужно разделять несколько задач: проверить корректность реализации, проверить устойчивость parser-a, проверить срабатывание каждого finding, проверить отсутствие шума на benign corpus, проверить dynamic collector на контролируемых сценариях, проверить отчёты, проверить производительность и оформить воспроизводимость.

Поэтому тестовая стратегия должна быть многоуровневой. Unit tests проверяют маленькие контракты кода. Synthetic regression binaries проверяют конкретные failure modes. Real benign corpus оценивает compatibility and false positive risk. Dynamic smoke проверяет runtime instrumentation. Full dynamic corpus проверяет, что dynamic reports сохраняют корректный contract на реальных executable outcomes. Fuzzing проверяет устойчивость к malformed inputs. Schema/semantic validation проверяет, что отчёты не просто создаются, а остаются машинно читаемыми и внутренне согласованными.

Уровень	Что проверяет	Почему нужен
Build/CTest	Сборка, unit tests, release gates.	Без воспроизводимой сборки нельзя говорить о готовности.
Unit tests	Safe reads, PE mapping, unwind decoder, heuristics, report escaping.	Ловят маленькие поломки близко к источнику.
Synthetic binaries	Каждый planned finding and edge case.	Доказывают, что правила вообще срабатывают на контролируемых нарушениях.
Real benign corpus	Совместимость с обычными PE and false positive risk.	Синтетика не показывает, насколько инструмент шумит на реальном мире.
Dynamic smoke	VEH, VCH, UEF, RtlAdd/RtlInstall function table paths.	Static tests не проверяют Windows Debug API collector.
Mixed scenarios	Static finding + runtime evidence correlation.	Показывают ценность связки static/dynamic.
Fuzzing/mutations	Malformed input robustness.	Недоверенный PE может быть специально создан для parser crash.
Schema/semantic checks	JSON/HTML/report invariants.	Отчёт является contract, а не просто красивой страницей.

12. Сколько бинарников нужно прогнать

Одного универсального числа не существует. Число зависит от того, какое утверждение защищается. Если утверждение “все 27 findings умеют срабатывать”, достаточно synthetic corpus с покрытием каждого finding и нескольких combination cases. Если утверждение “инструмент не шумит на нормальных PE”, нужен real benign corpus. Если утверждение “dynamic collector работает”, нужны dynamic smoke targets and real dynamic runs. Если утверждение “это production-quality tool”, нужны сотни или тысячи samples и длительный fuzzing.

Для ВКР разумно использовать ступенчатую шкалу достаточности. Минимум для демонстрации - 1-2 synthetic бинарника на каждый finding плюс clean controls. Уверенный уровень для защиты - 50-100 synthetic/mutated samples, не менее 100 real benign PE, 4-10 focused dynamic smoke targets and 10-20 mixed scenarios. Более сильный release-level уровень - 100-150 synthetic anomaly samples, 300-500 real benign PE из разных toolchains, 30-50 dynamic/mixed scenarios, 50-100 suspicious/protected samples при наличии безопасной изоляции, плюс fuzzing.

Текущий проект уже имеет неплохую основу: в папке “бинарники на тестирование” обнаружено 78 демонстрационных PE-бинарников: 60 static, 9 dynamic, 9 mixed. Отдельно присутствует real_samples corpus на 106 файлов. В evidence-сводке проекта также указаны gates: MinGW-w64 Release CTest 16/16 PASS, MSVC Release CTest 16/16 PASS, Real benign static corpus 106/106 PASS, Labeled static corpus 14/14 PASS, Dynamic smoke 4/4 PASS, Full dynamic corpus 124/124 valid, unexpected dynamic failures 0. Это можно защищать как хороший демонстрационно-регрессионный и validation evidence set, но не как абсолютное доказательство точности на всех PE-файлах мира.

Уровень заявления	Рекомендуемый объём	Что можно честно утверждать
Демонстрация механики	27-40 synthetic samples: минимум 1 на finding/семейство + clean controls.	Правила реализованы и умеют срабатывать на контролируемых нарушениях.
Нормальная защита ВКР	50-100 synthetic; 100+ benign real; 4-10 dynamic smoke; 10-20 mixed.	Проект покрывает заявленные признаки, не шумит на выбранном benign corpus, dynamic работает на подготовленных сценариях.
Сильная release validation	100-150 synthetic; 300-500 benign; 30-50 dynamic/mixed; 50-100 suspicious/protected safely collected.	Инструмент имеет серьёзную evidence-базу и пригоден к практическому экспертному использованию.
Product-grade research	1000+ benign; 200+ unusual benign; 100+ suspicious/protected; nightly fuzzing.	Можно обсуждать устойчивую оценку FP/FN по corpus, но всё равно не абсолютную гарантию.

13. Достаточно ли синтетических бинарников

Синтетические бинарники необходимы, но недостаточны. Они необходимы, потому что только synthetic samples позволяют точно контролировать нарушение: “здесь сломан CountOfCodes”, “здесь handler RVA ведёт в writable section”, “здесь RUNTIME_FUNCTION ranges overlap”, “здесь chain loop”. Без synthetic corpus трудно доказать, что каждый finding вообще проверен.

Но synthetic corpus не показывает реальную частоту ложных срабатываний. Реальные benign PE могут иметь особенности compiler/linker/runtime, которые не были заложены в синтетику: unusual section names, generated thunks, reused unwind info, large function tables, runtime library handlers, installer behavior, elevation requirements, DLL/PYD inputs. Поэтому для защиты качества обязательно нужен real benign corpus.

Также synthetic corpus не доказывает качество на вредоносных программах. Если проект не заявляет себя malware classifier, это нормально. Но если комиссия спрашивает про вредоносы, правильный ответ: “Для безопасной оценки на malicious corpus нужен изолированный стенд, curated samples, legal/ethical handling, static-first policy and no-network VM. В рамках текущего проекта malicious corpus является дополнительной validation stage, а не обязательной основой корректности EH/Unwind parser-a”.

Тип corpus	Что доказывает	Чего не доказывает
Synthetic	Покрытие правил, regression, edge cases, expected finding behavior.	Реальную false positive rate и поведение на живом разнообразии toolchains.
Real benign	Совместимость с нормальными файлами и риск шума.	Обнаружение всех suspicious/malicious EH/Unwind tricks.
Dynamic smoke	Collector видит контролируемые runtime events.	Покрытие всех ветвей реальной программы.
Mixed synthetic	Static/dynamic correlation работает на понятных сценариях.	Что correlation всегда будет доступна на реальных samples.
Real suspicious/malicious	Практическую применимость в security research.	Абсолютную точность или юридически безопасный universal benchmark.

14. Как строить synthetic corpus правильно

Synthetic corpus должен строиться не по принципу “много файлов”, а по принципу traceability: каждый бинарник имеет назначение, ожидаемый finding, ожидаемый score band, expected verdict, expected affected_count or representative condition, expected parse_error/dynamic event and documented mutation. Если файл сломан, должно быть понятно, что именно сломано и почему инструмент обязан это увидеть.

Для 27 findings минимальный профессиональный план - не менее одного positive sample на каждый finding и не менее одного negative control на каждую группу, где возможен benign аналог. Лучше иметь 2-3 samples на важные high severity findings: один простой isolated case, один boundary case, один combo/correlation case. Для менее важных findings допустимо по одному focused sample плюс inclusion in combo.

Нельзя делать все synthetic samples одинаковыми. Нужно покрывать разные уровни: PE container faults, RUNTIME_FUNCTION faults, UNWIND_INFO decode faults, metadata placement faults, reuse/dominance distribution patterns, strict diagnostics, handler/chained info cases. Отдельно нужны clean binaries: минимальный clean, richer clean, many-function clean, unusual but allowed section clean, no runtime functions clean if supported by scope. Они нужны, чтобы показать, что проект не ругается на всё подряд.

Группа	Минимум	Профессионально
Clean controls	3-5	8-12: minimal, richer, many functions, .rdata/.xdata variants, no-anomaly dynamic.
27 finding coverage	27	54-81: 2-3 на каждый важный finding, включая boundary cases.
Combo/correlation	5-8	15-25: placement+integrity, reuse+dominance, static+dynamic, high suspicion full combo.
Dynamic focused	4	10-20: VEH, VCH, UEF, RtlAdd, RtlInstall, timeout, no signal, multi handler, handler hit.
Mixed	4-6	10-20: static anomaly + runtime signal, static anomaly without route, clean static + runtime table.

15. Как строить real benign corpus

Real benign corpus нужен для оценки compatibility. Он должен состоять из легально полученных, известных нормальных Windows x64 PE-файлов: утилиты Microsoft/Sysinternals, официальные CLI tools, Python/Rust/Go/C++ binaries, DLL/PYD modules, installers, normal GUI/console apps. Цель - проверить, что анализатор не создаёт лишние findings на обычной compiler-generated EH/Unwind metadata.

Корпус должен быть разнообразным. Если взять 100 файлов только из одного набора, например одного toolchain, результат будет слабее. Нужно включать разные компиляторы, размеры, типы исполняемых файлов, DLL and EXE, файлы с большим

числом RUNTIME_FUNCTION entries and files with small tables. Для dynamic evaluation нужно различать launchable EXE, DLL/PYD that cannot be launched by CreateProcess, installers requiring elevation and long-running tools.

Для ВКР текущие 106 real samples - хороший baseline. Они позволяют сказать: “на выбранном реальном benign corpus отчёты валидны и не было non-zero heuristic score”. Но нельзя говорить: “false positive rate равен 0 во всём мире”. Корректно: “observed false positive rate on this corpus is 0/106, with stated corpus composition and limits”.

Размер benign corpus	Защитная сила
30-50	Только первичная smoke compatibility. Для ВКР слабовато, но лучше нуля.
100-150	Нормально для ВКР: уже можно говорить о выбранном реальном compatibility corpus.
300-500	Сильный release-level аргумент, если источники разнообразны.
1000+	Product/research level; можно осторожно обсуждать FP-rate по corpus and toolchain categories.

16. Как строить suspicious/protected corpus

Suspicious/protected corpus нужен только если проект хочет сильнее показать практическую применимость в security research. Он не обязателен для доказательства корректности parser-a, потому что parser/decoder rules проверяются synthetic and unit tests. Но он полезен для демонстрации того, что EH/Unwind findings встречаются не только в искусственных samples.

Важно не смешивать suspicious corpus и malware classification. Если файл вредоносный, проект всё равно анализирует не “вредоносность вообще”, а EH/Unwind layer. В отчёте нужно писать: sample has EH/Unwind anomalies, runtime handler activity or dynamic function table evidence. Не писать: analyzer proved malware. Такой корпус должен запускаться только в isolated Windows VM, preferably offline or network-simulated, with snapshots and controlled privileges.

Для ВКР достаточно указать planned/optional stage: 20-50 suspicious/protected samples under isolation. Для product-level можно расширять до 100+. Но если нет безопасного стенда и юридически чистого доступа к samples, лучше не обещать такую проверку как выполненную.

> Если комиссия спросит “как анализировать вредоносы?”, ответ: static можно делать без запуска; dynamic только в изолированной Windows VM/sandbox, со снапшотом, без доверенной сети, с контролем времени и без работы на основной машине.

17. Метрики качества: что считать точностью

Для этого проекта нельзя сводить качество к одной accuracy, как в обычной бинарной классификации. Проект не является ML-classifier and does not output malware/benign probability. Поэтому качество оценивается по нескольким группам метрик: parser robustness, finding coverage, report validity, benign compatibility, dynamic signal detection, performance and reproducibility.

True positive в synthetic context означает: sample был создан с известным нарушением, и expected finding появился. False negative в synthetic context означает: expected finding не появился. False positive on benign corpus означает: normal benign sample получил non-zero score or unexpected finding. True negative означает: clean/benign sample прошёл без finding в рамках expected behavior. Но эти понятия надо применять к конкретной задаче: “finding-level TP/FP”, а не “malware-level TP/FP”.

Если комиссия спрашивает “какова точность модели”, ответ: “В проекте нет статистической ML-модели, поэтому корректнее говорить не об одной accuracy, а о покрытии правил и наблюдаемой ошибочности на corpus. Для synthetic labeled corpus измеряется finding recall: сколько ожидаемых findings сработало. Для benign corpus измеряется observed false positive rate. Для dynamic smoke измеряется expected signal detection rate. Для reports измеряется schema/semantic validity”.

Метрика	Как считается	Как защищать
Finding recall on synthetic	Expected findings detected / expected findings.	Показывает, что правила срабатывают на известных нарушениях.
Observed FP on benign	Benign files with unexpected non-zero score / benign files.	Показывает шум на выбранном real corpus, не на всём мире.
Report validity	Schema-valid and semantic-valid reports / all reports.	Показывает стабильность output contract.
Dynamic detection	Expected runtime signals captured / prepared dynamic scenarios.	Показывает работу collector на контролируемых paths.
Robustness	Crashes, OOB, invalid reports under malformed/fuzz inputs.	Для parser недоверенных bytes это критично.
Performance	Median, p95, max analysis time; entries/sec.	Показывает пригодность для batch triage.

18. Что уже есть в текущем проекте как evidence

По текущей структуре проекта можно защищать наличие нескольких уровней evidence. В папке “бинарники на тестирование” обнаружено 78 демонстрационных PE-бинарников: 60 static, 9 dynamic and 9 mixed. Это полезный showcase corpus: он показывает разные статические, динамические и смешанные ситуации, а также даёт HTML/JSON examples для комиссии.

Отдельно в проекте есть real_samples/inbox на 106 файлов. В summary real static corpus указано: 106 files, 106 successful analyses with valid reports, 0 failed/timed out/invalid reports, 0 files with non-zero heuristic score, 174,795 parsed RUNTIME_FUNCTION records, total input size 140.4 MiB, median analysis time 17 ms, p95 36 ms, max 268 ms. Это сильный compatibility аргумент для ВКР.

В финальной evidence-сводке проекта указаны gates: MinGW-w64 Release CTest 16/16 PASS, MSVC Release CTest 16/16 PASS, Real benign static corpus 106/106 PASS, Labeled static corpus 14/14 PASS, Dynamic smoke 4/4 PASS, Full dynamic corpus 124/124 valid, Dynamic evidence validator PASS, Unexpected dynamic failures 0. Эти числа важно подавать честно: они подтверждают конкретные validation gates, но не являются абсолютной гарантией на всех возможных PE-файлах.

Evidence в текущем проекте	Как правильно защищать
78 демонстрационных PE	Showcase corpus: удобно показывать static/dynamic/mixed отчёты и покрытие сценариев.
106 real benign samples	Compatibility corpus: observed 0/106 non-zero scores in this selected corpus.
16/16 CTest MinGW and 16/16 CTest MSVC	Build/test gates для двух Windows toolchain configurations.
4/4 dynamic smoke	Контролируемая проверка VEH/VCH/UEF/RtlFunctionTable paths.
124/124 dynamic reports valid	Dynamic report contract работает на полном корпусе outcomes.
0 unexpected dynamic failures	Ошибки dynamic corpus классифицированы как expected outcomes: timeout/elevation/unsupported library etc.

19. Что ещё желательно добавить для максимально сильной защиты

Если цель - не просто сдать ВКР, а выглядеть максимально уверенно перед жёсткой комиссией, текущую evidence-базу можно усилить не переписыванием ядра, а методологией и дополнительными прогонами. Самое важное - расширить labeled static corpus так, чтобы каждый из 27 findings имел минимум один explicit sample and expected JSON assertion. Сейчас демонстрационный pack шире, но formal labeled validation в summary указан как 14 files. Для защиты хорошо сказать: “демо-бинарников 78, но formal labeled gate следует расширить до full 27-finding matrix”.

Второе усиление - benign corpus. 106 файлов уже хорошо. Сильнее будет 300-500 real benign PE из разных источников: Microsoft/Sysinternals, Windows SDK tools, Python distribution, LLVM/MinGW/MSVC-built examples, Rust/Go binaries, normal DLLs, GUI and console apps. Главное - хранить provenance, hash and source category.

Третье усиление - dynamic and mixed scenarios. Для dynamic достаточно не множества случайных EXE, а набора, который покрывает event types and outcome types: clean no signal, VEH registration/hit, VCH, UEF, RtlAddFunctionTable, RtlInstallFunctionTableCallback, timeout, elevation-required, unsupported DLL, multi-handler, handler hit with static map, static finding without runtime hit. Для ВКР 10-20 таких focused сценариев выглядят профессионально.

Четвёртое усиление - fuzzing. Для C parser-а недоверенных PE это особенно важно. Минимально можно запускать mutation fuzz на seed corpus и фиксировать crash-free run. Хороший уровень - 30-60 минут sanitizer-enabled fuzzing или 100,000+ mutations без crash/invalid memory access. Product-level - nightly fuzzing in CI.

Что усилить	Минимально	Сильно
Formal 27-finding matrix	27 samples + expected finding assertions.	54-81 samples, по 2-3 на важные findings and boundary cases.
Benign corpus	100-150 files.	300-500 files from diverse toolchains/sources.
Dynamic focused tests	4 smoke scenarios.	10-20 focused scenarios plus 30-50 real dynamic runs.
Mixed correlation	5 examples.	10-20 examples with correlation/no-correlation/incomplete cases.
Fuzzing	10k-50k mutations or short fuzz run.	100k+ mutations or 30-60 min sanitizer fuzz; nightly for product.

20. Пошаговый протокол тестирования перед защитой

1. Зафиксировать версию проекта: tool version, schema version, SHA-256 executable, дата сборки, compiler/toolchain.
2. Собрать Release под MinGW-w64 и MSVC, включить warnings-as-errors where applicable, сохранить build logs.
3. Запустить CTest and unit tests. В отчёте указать не просто “прошло”, а количество gates: например 16/16 PASS.
4. Запустить synthetic regression corpus. Для каждого sample проверить expected finding, expected verdict/score band, JSON validity and HTML generation.
5. Запустить 27-finding matrix: убедиться, что каждый finding имеет хотя бы один positive sample and one documented explanation.
6. Запустить clean synthetic controls: minimal clean, richer clean, many-function clean, allowed unusual section clean, clean dynamic no signal.

7 Запустить real benign static corpus. Посчитать files, valid reports, failed reports, non-zero scores, parsed runtime functions, time median/p95/max.

8 Запустить dynamic smoke на Windows x64 VM: VEH, VCH, UEF, RtlAdd/RtlInstall function table. Проверить expected events.

9 Запустить full dynamic corpus с timeout and trace budget. Разделить outcomes: completed, controlled_timeout, elevation_required, unsupported_library_launch, unexpected_failure.

10 Запустить report validators: JSON schema, semantic invariants, dynamic evidence validator, HTML presence check.

11 Запустить mutation fuzz/parser robustness на seed corpus. Зафиксировать duration or mutation count, crashes=0, sanitizer issues=0.

12 Собрать итоговый SUMMARY_RU: что проверено, сколько файлов, какие outcomes, какие ограничения, что не утверждается.

21. Как оформить тестовые бинарники

Каждый тестовый бинарник должен иметь паспорт. Без паспорта набор превращается в папку с непонятными exe. Паспорт нужен не только тебе, но и комиссии: он показывает, что бинарники не случайны, а построены по методике.

Паспорт должен содержать: ID, filename, category static/dynamic/mixed/clean/real, intended purpose, what is broken, expected finding codes, expected severity/verdict/score band, expected dynamic events if any, expected report fields, source/generator script, build toolchain, SHA-256, safety notes and manual explanation. Если бинарник synthetic, нужно писать, какая именно структура нарушена: Exception Directory size, RUNTIME_FUNCTION range, UnwindInfoAddress, CountOfCodes, UWOP operands, handler RVA, section permission, chain path.

```
Recommended manifest fields:
{
"id": "static_uwop_slots_beyond_count",
"file": "77_static_diag_uwop_slots_beyond_count.exe",
"category": "static",
"purpose": "[redacted], [redacted] decoder [redacted] [redacted] UWOP operands [redacted] CountOfCodes.",
"mutation": "CountOfCodes [redacted] [redacted] [redacted] [redacted] [redacted] slots.",
"expected_findings": ["unwind_parse_fail"],
"expected_fields": ["parse_error", "representative_rva", "affected_count"],
"expected_verdict_band": "low/moderate/high according to score policy",
"safety": "Synthetic PE, static-only safe; dynamic optional in VM."
}
```

22. Как отвечать комиссии по тестам

Вопрос комиссии	Профессиональный ответ
Сколько бинарников достаточно?	Зависит от утверждения. Для finding coverage нужен минимум 1 sample на finding. Для ВКР лучше 50-100 synthetic + 100+ benign real + dynamic smoke/mixed. Для product-level нужны сотни benign and suspicious samples.
Хватит ли синтетических?	Нет, одних synthetic недостаточно. Они доказывают срабатывание правил, но не показывают false positive behavior на реальных benign PE. Поэтому нужен real benign corpus.
Нужно ли запускать реальные вредоносы?	Не обязательно для parser correctness. Для security relevance можно добавить isolated malicious/protected corpus, но только в VM/sandbox. Проект не является malware classifier.
Почему 16 CTest, если бинарников 78?	CTest - это gates/unit/integration checks. 78 бинарников - showcase/regression corpus. Это разные уровни тестирования, они не обязаны совпадать по числу.
Что значит точность?	Не одна ML accuracy. Используются finding recall on synthetic, observed FP on benign, dynamic signal detection, report validity, crash-free fuzzing and performance metrics.
Если на 106 benign FP=0, можно сказать FP=0?	Можно сказать observed FP=0/106 on selected corpus. Нельзя утверждать FP=0 во всём мире.
Почему dynamic corpus имеет timeouts/elevation?	Это нормальные outcome classes. Важно, что analyzer корректно оформляет controlled_timeout/elevation_required/unsupported_library_launch, а unexpected failures равны 0.

23. Какой объём тестирования защищает разные уровни готовности

Для финальной ВКР достаточно не product-grade миллиона samples, а убедительной, воспроизводимой и честной evidence-базы. Комиссия должна увидеть, что покрыты три вопроса: работает ли код, срабатывают ли признаки, не шумит ли инструмент на нормальных файлах. Для этого текущий набор уже выглядит неплохо, но документ должен честно отделять выполненное от recommended next step.

Для “полноценного релиза заказчику” лучше иметь более строгий release checklist: formal 27-finding matrix, 300+ benign PE, repeatable dynamic smoke, report schema validation, fuzzing logs, exact tool hash, release notes, known limitations. Это не значит, что без 300 benign проект бесполезен; это значит, что большее заявление требует большей evidence-базы.

Уровень	Минимальная evidence-база	Формулировка готовности
Учебная демонстрация	Unit tests + несколько synthetic + HTML/JSON examples.	Показывает принцип работы, но слабовато для жёсткой защиты качества.
Защита ВКР	CTest, 50-100 synthetic/showcase, 100+ benign real, 4+ dynamic smoke, schema validation.	Достаточно для аргументированной защиты заявленного scope.
Заказчик / release	Full finding matrix, 300-500 benign, 30-50 dynamic/mixed, fuzzing, release package and docs.	Можно передавать как экспертный инструмент с known limitations.
Исследовательский продукт	1000+ benign, suspicious/protected corpus, nightly fuzzing, CI metrics, versioned datasets.	Можно обсуждать устойчивые quality metrics by corpus categories.

24. Итоговая позиция по текущему проекту

Текущий проект можно защищать как специализированный, завершённый в своём scope инструмент для static EH/Unwind analysis with optional Windows dynamic validation and reporting. Его сильные стороны: узкая, но реальная low-level область; объяснимые findings; structured reports; Ghidra bridge; наличие synthetic/dynamic/mixed showcase; real benign corpus; dynamic validation summaries; разделение static and runtime evidence.

При этом профессионально нужно сказать: для максимального product-grade заявления тестовую базу стоит расширять. Особенно полезно формализовать 27-finding matrix в отдельный labeled gate, увеличить benign corpus до 300-500 для release-level аргумента, добавить больше mixed correlation scenarios and preserve fuzzing logs. Это не отменяет готовность текущего ядра, но делает защиту качества значительно сильнее.

Главная защита: проект не обещает невозможного. Он не утверждает “мы нашли все вредоносы”. Он утверждает: “мы построили инструмент, который проверяет Windows x64 EH/Unwind metadata, фиксирует структурные anomalies, помогает реверсеру не ошибиться в ручной работе, даёт воспроизводимые отчёты и показывает runtime evidence там, где static не может видеть процесс”. Это профессиональная и честная позиция.

25. Источники и опора документа

- Глава 11 книги “Reverse Engineering Windows x64 - final no-loss v89”: архитектура EH/Unwind Analyzer, назначение static/dynamic слоёв, reporting layer, Ghidra integration, validation/corpus/fuzzing and final architecture.
- Microsoft Learn: x64 exception handling - Function Unwind Information, RUNTIME_FUNCTION, UNWIND_INFO, handler flags, chained unwind info and dynamic function tables.
- Microsoft Learn: PE Format - Exception Directory and .pdata section as function table entries used for exception handling.
- Microsoft Learn: RtlVirtualUnwind - восстановление invocation context предыдущей функции по unwind metadata.
- Microsoft Learn: RtlAddFunctionTable and RtlInstallFunctionTableCallback - dynamic function tables for dynamically generated code.
- Microsoft Learn: WaitForDebugEvent, ContinueDebugEvent and Debugging Events - основа Windows debugger loop used by dynamic validation.
- Ghidra documentation - comments, bookmarks, scripting and workflow integration for manual reverse engineering.
- Локальные evidence-файлы проекта: reports/final_validation_2026-06-19/SUMMARY_RU.md, static_real/SUMMARY.md, static_labeled/SUMMARY.md, dynamic_full/REPORT.md, папка “бинарники на тестирование”.

```
Reference URLs:
https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64?view=msvc-170
https://learn.microsoft.com/en-us/windows/win32/debug/pe-format
https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtlvirtualunwind
https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtladdfunctiontable
https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtlinstallfunctiontablecallback
https://learn.microsoft.com/en-us/windows/win32/api/debugapi/nf-debugapi-waitfordebugevent
https://learn.microsoft.com/en-us/windows/win32/api/debugapi/nf-debugapi-continuedebugevent
https://learn.microsoft.com/en-us/windows/win32/debug/debugging-events
https://ghidra.re/ghidra_docs/GhidraClass/Beginner/Introduction_to_Ghidra_Student_Guide.html
```

ДОПОЛНЕНИЕ: КАРТА EH/UNWIND И ПРОФЕССИОНАЛЬНОЕ ТЕСТИРОВАНИЕ

Раздел добавлен к документу “ОБОСНОВАНИЕ ПРОЕКТА”: он раскрывает мысль из книги о том, что проект не просто парсит PE, а строит карту, которую затем читает реверс-инженер, и уточняет методику проверки качества.

A. Что означает построение карты

Построение карты в данном проекте означает перевод недоверенного PE-файла из набора байтов в связанную модель объектов EH/Unwind. Parser читает структуру. Карта связывает структуры между собой и показывает, как по ним должен двигаться анализ. В карте есть координаты, узлы, связи, доказательства, нарушения и точки ручной проверки. Поэтому результат проекта является не просто таблицей полей, а ориентиром для дальнейшей reverse engineering работы.

Карта нужна потому, что EH/Unwind-слой сам по себе распределён по нескольким областям. Exception Directory находится в DataDirectory. Таблица RUNTIME_FUNCTION обычно находится в .pdata. Записи указывают на UNWIND_INFO, который обычно лежит в .xdata. Handler RVA может вести в кодовую секцию, в runtime stub или в необычную область. Language-specific handler data находится после handler RVA, но её формат определяется конкретным handler/runtime. Dynamic function table может появиться только во время выполнения. Без карты реверсер видит разрозненные адреса; с картой он видит систему связей.

Главная защитная формулировка: проект строит карту обещаний файла и, при dynamic run, карту наблюдаемого исполнения. Static layer отвечает на вопрос: “какую EH/Unwind-карту заявляет PE на диске?”. Dynamic layer отвечает на вопрос: “какие exception/unwind-related события реально появились в процессе?”. Эти две карты не заменяют друг друга. Они позволяют специалисту сравнить file-backed evidence and runtime evidence.

Слой карты	Что фиксируется	Зачем это реверсеру
Адресная карта	file offset, RVA, VA, ImageBase, section mapping.	Не перепутать физическое положение байтов, image-relative адреса и адреса в памяти. Ошибка на этом уровне делает весь дальнейший вывод ненадёжным.
Карта секций	Имена секций, raw/virtual ranges, readable/writable/executable permissions.	Понять, нормально ли расположены .pdata, .xdata, handler targets and metadata; увидеть suspicious placement.
Карта RUNTIME_FUNCTION	BeginAddress, EndAddress, UnwindInfoAddress, сортировка, overlaps, duplicates, gaps.	Получить interval map функций и увидеть, не создаёт ли таблица неоднозначность для ControlPc.
Карта UNWIND_INFO	Version, Flags, SizeOfProlog, CountOfCodes, UWOP slots, padding, handler/chained tail.	Понять, можно ли безопасно интерпретировать unwind metadata и где чтение должно остановиться из-за malformed данных.
Карта handlers/chains	EHANDLER, UHANDLER, CHAININFO, handler RVA, chained records, handler data boundary.	Не спутать handler RVA, chained info and language-specific data; увидеть possible exception route.
Слой findings	family, severity, confidence, affected objects, representative RVA, explanation.	Понять не только “что сработало”, но и где, почему, насколько уверенно и что проверять руками.
Dynamic evidence map	debug events, exceptions, handler hits, runtime function table registrations.	Связать static hypothesis с фактическим runtime behavior в конкретном запуске.

B. Почему карта профессиональнее простого parser output

Простой parser output может сообщить: RUNTIME_FUNCTION[42], BeginAddress = 0x12000, EndAddress = 0x12380, UnwindInfoAddress = 0x34020. Это полезно, но недостаточно. Профессиональный анализ должен объяснять отношения: этот range находится в .text; unwind info находится в .xdata; запись участвует в отсортированной function table; декодирование UNWIND_INFO успешно или оборвалось на конкретном поле; handler target расположен в такой-то секции; finding относится к такому-то семейству; dynamic run подтвердил или не подтвердил route.

Именно это и называется картой. Карта даёт проверяемый маршрут PE -> Exception Directory -> RUNTIME_FUNCTION -> UNWIND_INFO -> handler/chained info -> finding -> manual validation. Такой маршрут можно открыть в HTML, сохранить в JSON, передать backend, импортировать в Ghidra and использовать как план ручного анализа. В этом смысле проект является системой структурного анализа: parser является фундаментом, но ценность продукта появляется на уровне связей и evidence.

На защите можно сказать так: “Если parser отвечает ‘какие поля прочитаны’, то мой проект отвечает ‘как эти поля образуют EH/Unwind-карту файла, где карта противоречива, где она необычна и какие места должен проверить специалист’. Поэтому термин analyzer корректнее, чем parser”.

C. Как реверс-инженер читает карту

Работа реверсера с картой начинается с общей сводки. Он смотрит количество записей, наличие Exception Directory, parse status, количество findings, общий verdict and confidence. Это даёт первичную сортировку: файл обычный, файл требует ручной проверки или файл содержит серьёзные нарушения EH/Unwind-структур.

Далее специалист переходит к конкретным адресам. Он проверяет section map: где находится .pdata, где находится .xdata, где лежит handler target, есть ли writable/executable context. Затем он смотрит function range map: BeginAddress < EndAddress, нет ли overlaps and duplicate begins, попадает ли ControlPc или интересующий RVA внутрь диапазона. Затем он смотрит decoded UNWIND_INFO: version, flags, prolog size, CountOfCodes, UWOP operations, extra slots and padding. Если декодирование обрывается, карта показывает не просто “ошибка”, а уровень, на котором нарушился контракт.

После этого реверсер решает, нужна ли динамика. Если static finding связан с handler route или dynamic function table suspicion, запуск в isolated Windows VM может дать runtime evidence. Dynamic output не отменяет static finding автоматически: если путь не был достигнут, отсутствие handler hit не означает, что route невозможен. Но если handler hit или RtlAddFunctionTable event наблюдался, карта становится сильнее: static evidence связывается с реальным runtime событием.

D. Как правильно тестировать такую карту

Тестирование проекта должно проверять не только “запускается ли программа”. Нужно проверять, что карта строится правильно на каждом уровне. Для адресной карты нужны samples с корректным и некорректным RVA mapping. Для section map нужны cases с нормальными и необычными section permissions. Для RUNTIME_FUNCTION map нужны sorted table, unsorted table, overlap, duplicate begin, invalid begin/end, directory size not multiple of 12. Для UNWIND_INFO map нужны valid simple record, odd CountOfCodes with padding, UWOP requiring extra slots, truncated code array, invalid version, unsupported or malformed flags, bad handler tail, chain cycle. Для handler map нужны normal __C_specific_handler-like case, unknown handler, handler in unusual section, handler dominance and reuse. Для dynamic map нужны controlled VEH/VCH/UEF and dynamic function table registration cases.

Поэтому количество тестовых бинарников должно выводиться из матрицы покрытия, а не из красивого числа. Если в проекте 27 признаков, это не значит, что нужно ровно 27 тестов. Один sample может покрывать несколько признаков, а один важный признак может требовать несколько variants: normal, boundary, malformed, combined. Для сильной защиты лучше иметь больше samples, чем findings, и у каждого файла должен быть паспорт с expected observations.

Синтетические samples дают ground truth и обязательны для positive testing. Real benign corpus нужен для false positive pressure. Dynamic samples нужны для runtime visibility. Mixed samples нужны для проверки связки static hypothesis -> dynamic evidence. Suspicious/protected samples можно использовать только в isolated VM и только как демонстрацию triage, а не как статистическое доказательство malware detection.

Что проверяется	Какие binaries нужны	Какой вывод можно защищать
Корректность parsing/decoding	Минимальные synthetic PE для каждого формального случая.	Проект правильно реализует documented EH/Unwind contracts на контролируемых данных.
Покрытие эвристик	Labeled malformed/unusual samples по всем families findings.	Каждый признак имеет ожидаемый positive case и объяснимое evidence.
Низкий шум	Real benign corpus: системные утилиты, разные компиляторы, разные размеры.	Инструмент не превращает нормальное разнообразие PE в массовые high findings.
Dynamic validation	Windows-only smoke targets for exceptions, handlers and function table APIs.	Dynamic collector видит runtime events, которые static не может вывести из файла.
Связка static/dynamic	Mixed samples, где static finding проверяется исполнением.	Проект может строить две карты: file-backed and runtime-observed.

E. Как отвечать комиссии про объём тестов

Если комиссия спрашивает “сколько файлов достаточно”, правильный ответ: “Достаточность определяется coverage matrix”. Для учебной демонстрации хватит десятков файлов, если каждый подписан и показывает конкретный case. Для уверенной защиты ВКР разумная база — все unit/CTest gates, не менее 50-100 synthetic/showcase binaries, не менее 100 benign real PE, отдельный dynamic smoke набор and schema/semantic validation. Для уровня заказчика или release желательно расширять real benign corpus до 300-500+ файлов, добавить 30-50 dynamic/mixed cases, включить fuzzing and versioned regression reports.

Важно не обещать абсолютную точность. Проект не ML-classifier и не универсальный malware detector. Качество оценивается по конкретным задачам: functional pass rate, expected finding hit rate, report validity rate, false positive review on benign corpus, dynamic evidence pass rate, crash-free fuzzing. True positive and false negative считаются по конкретному finding, а не по общей вредоносности файла. Такая позиция звучит профессионально и защищаемо.

Итоговая формула: проект строит карту, а тесты должны доказывать корректность каждого слоя этой карты. Сначала проверяется чтение и адресация, затем интервалы RUNTIME_FUNCTION, затем UNWIND_INFO, затем handlers/chains,

затем findings, затем dynamic evidence and reports. Поэтому тестирование является не приложением к проекту, а способом доказать, что карта действительно достоверна.